

Team Run – Final Project

Team Run Group Members- Backend Team

1. Daniel Herrera - Backend / Cloud Lead
2. Osbaldo Bravo - System Integration Lead

Frontend Team

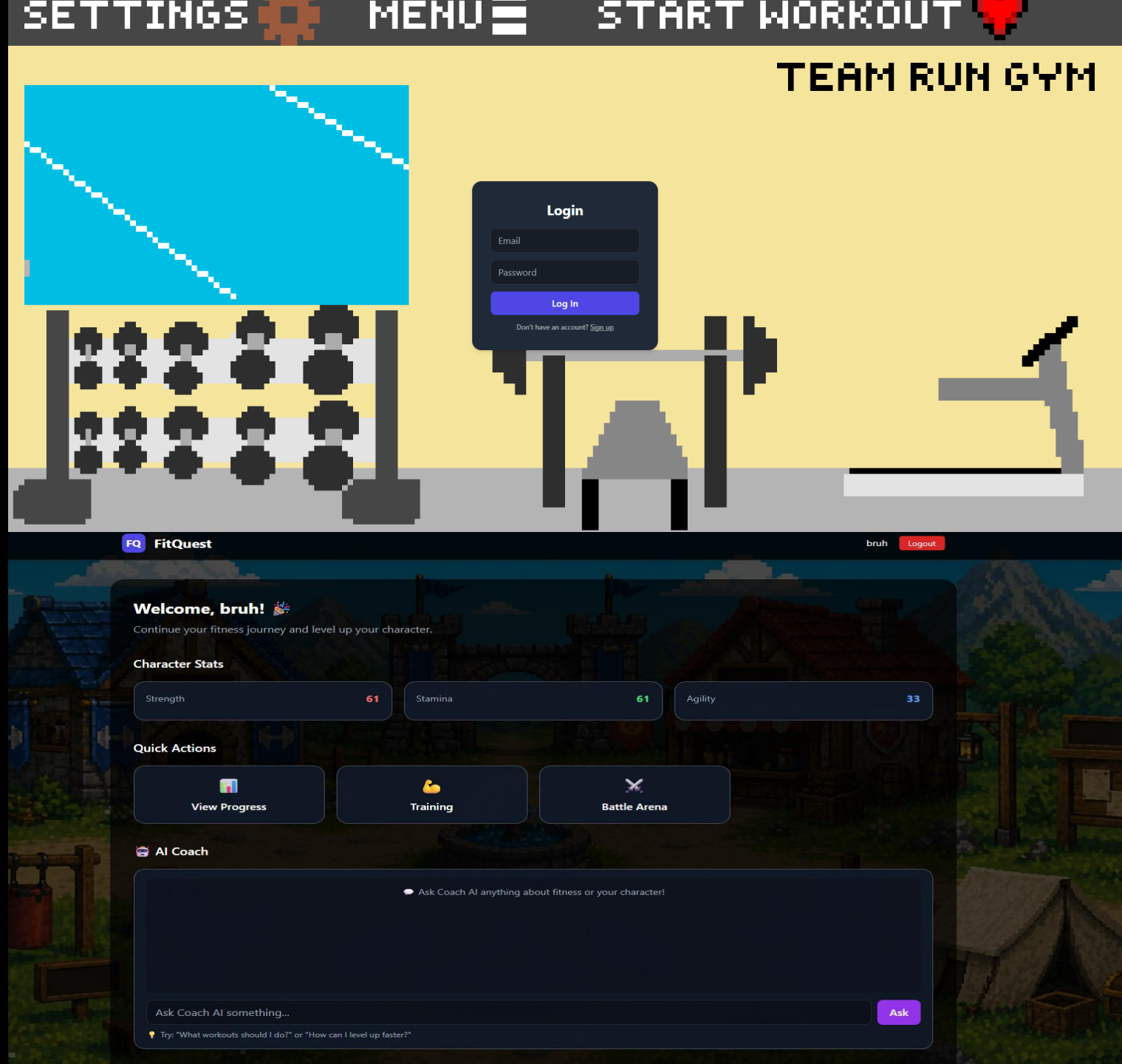
3. Carlos Martinez - QA & Documentation Lead
4. Anthony Jimenez - Frontend UI Developer
5. Harut Kulakchian – Frontend Asset/UI Developer

• View of The App

-Workout RPG Tracker is a fitness tracking web/mobile application that combines workout logging with role-playing game mechanics. Users log exercises such as pushups, squats, pull-ups, and hydration intake. Each activity updates character stats in real time.

-The system stores user data securely, calculates experience points (XP), and unlocks features such as boss battles, raids, customization, and progression zones based on consistency and performance.

-The goal is to increase workout motivation through a game while maintaining real fitness tracking accuracy.



Problem Statement

Some people are overwhelmed when it comes to starting or even continuing their fitness journey. Whether due to the overwhelming amount of machines and workouts there are, or just getting bored about going altogether. Team Run aimed to solve these issues with a fitness RPG app in order to answer questions users may have, guide users on their fitness journey, and give them a way to have fun while doing it

Stakeholders: Gym beginners, casual lifters, people who enjoy video games, students and those that are extremely busy. Others include fitness communities looking for other tools to expand, coaches looking for more engaging tools for their clients.

Technology Stack (updated)

Layer	Technology	Switched From
Frontend	HTML5, Tailwind CSS, JavaScript	N/A
Backend	Node.js, Express.js	N/A
Database	MongoDB Atlas	MongoDB Compass (Local)
AI	Groq (Llama 3.3 70B), Rule-based expert system	No AI implementation before
Cloud Hosting	Railway (Google Cloud Platform)	LocalHost



Character Animations



Workout Complete!

Your character celebrates your achievement



Strength Boost



Energy Surge



Health Increase



New Ability

Dynamic Exercise Animations

Your character responds to different workout types with specialized animations:



Weight Training

Character visibly strains with heavy weights



Cardio

Character runs faster as pace increases



Yoga

Character flows smoothly between poses

Animation Examples

- Pushups → Arm muscles flex
- Squats → Legs glow with power
- Cardio → Character runs in place
- New PR → Victory pose
- Level up → Full transformation

Motivational System

Animations provide visual feedback and encouragement to keep users engaged with their fitness journey.



Log Reps & Weight

Comprehensive Workout RPG System

Our advanced algorithm converts your physical efforts into character progression with:

Strength Training

Higher weights = more XP

Endurance

More reps = stamina bonuses

Cardio

Distance = movement speed

Flexibility

Yoga = agility stats

XP Calculation

$$\text{XP} = (\text{Reps} \times \text{Weight}) \times \text{Exercise Multiplier}$$

Different exercises have different multipliers based on intensity.

Progression Tracking

- Visual graphs of strength gains
- Milestone achievements
- Personal records tracking
- Workout history calendar

Log Workout



Bench Press

Weight (lbs)

-

135

+

Repetitions

-

8

+

Sets

-

3

+

XP Earned:

+324 XP



Log Workout



Water Intake Tracker

Hydration = Health

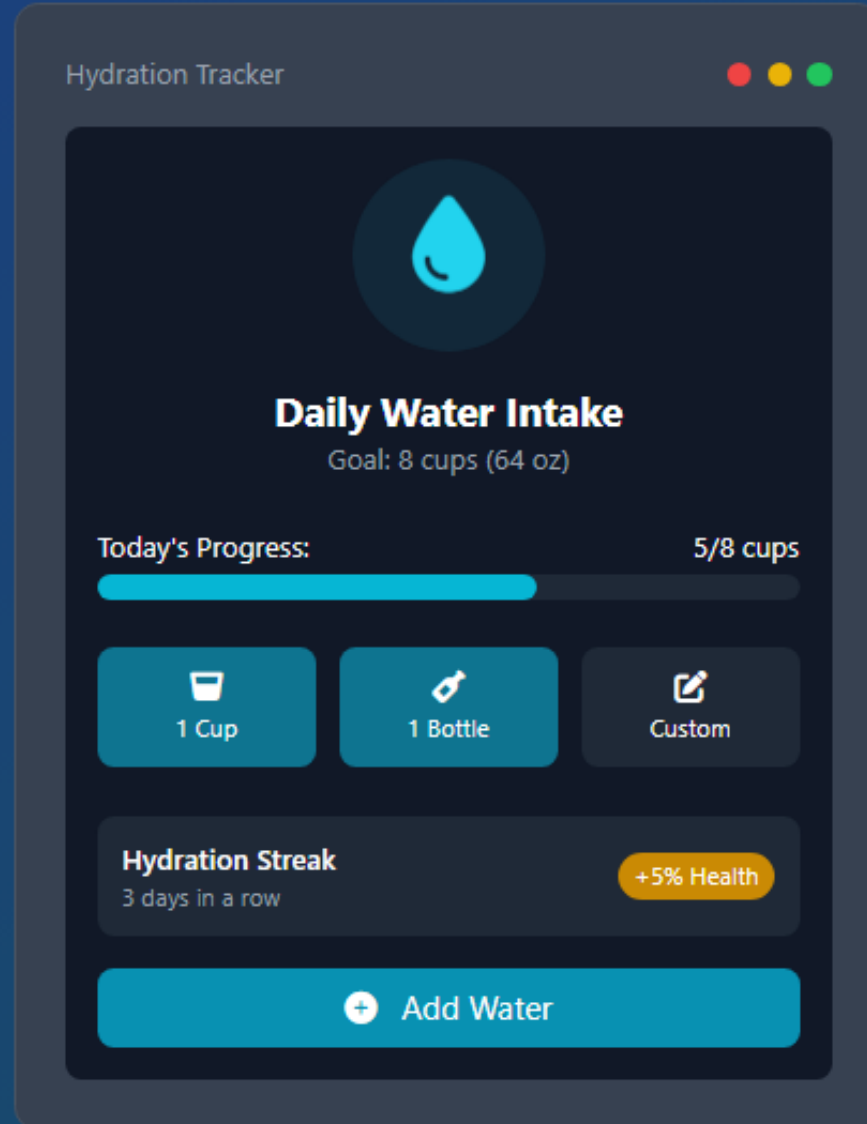
Track your water intake to boost your character's health and stamina stats.

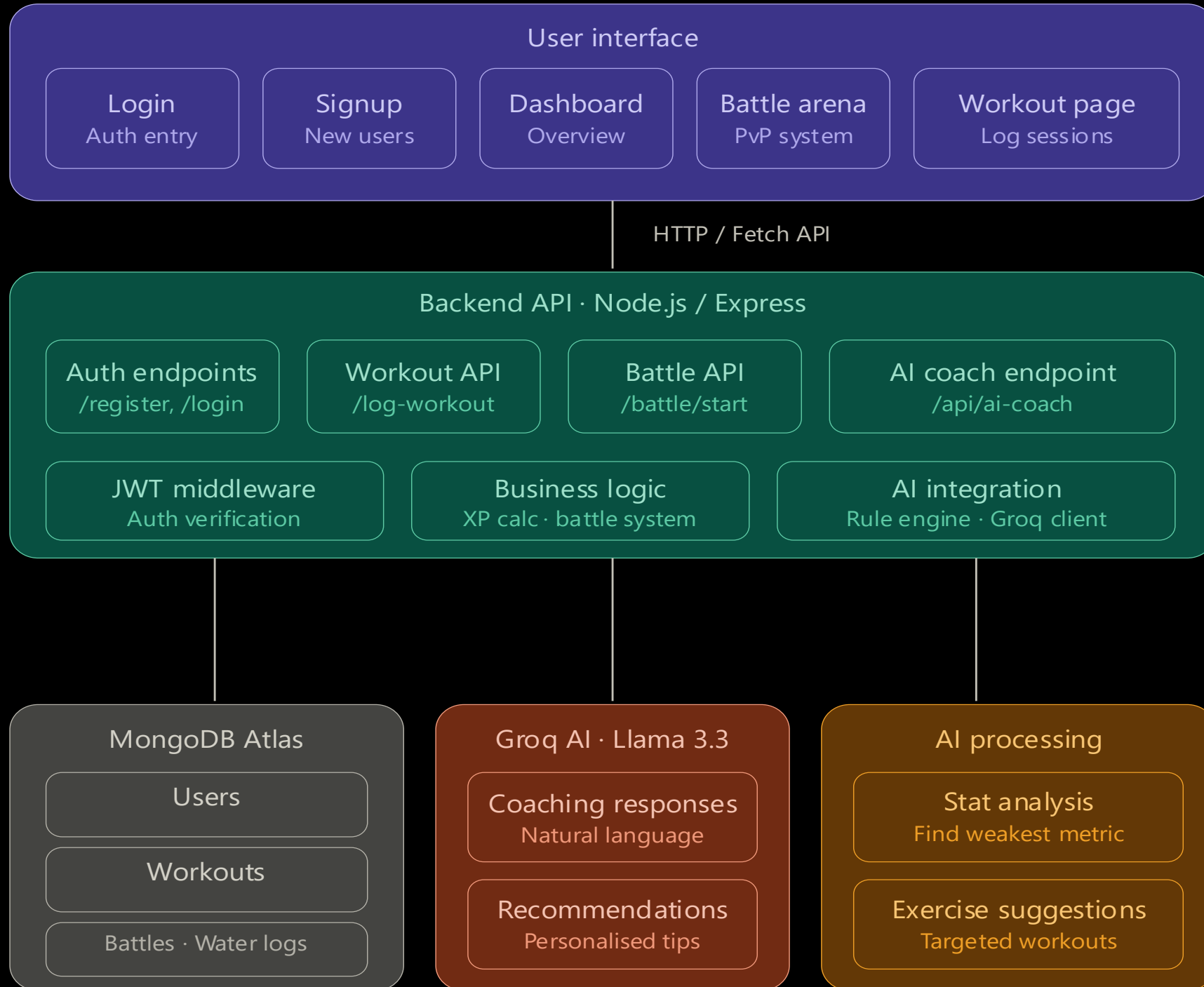
Features

- Daily water goal tracking
- Reminder notifications
- Hydration streak rewards
- Visual character effects when hydrated
- Health stat bonuses

Health Benefits

Proper hydration improves workout recovery and overall health, reflected in your character's abilities.





System Flow

UI (Frontend)

- This is where users interact with the app, including logging workouts, tracking hydration and participating in battles. It provides the visual interface and sends user actions to the backend.

API (Backend)

- This handles all incoming requests from the frontend, authenticates users and processes application logic such as workouts, battles and progression. It connects the frontend to the database and AI system.

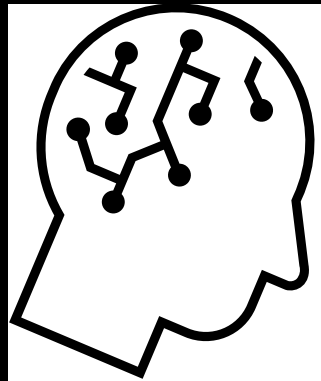
Database

- This stores all user related data including account information, workout logs, XP, coins and battle results. It ensures data is saved and can be retrieved efficiently when needed.

AI System (Cloud API)

- This generates AI-based responses such as workout recommendations and guidance. It enhances the user experience by providing intelligent feedback based on user input.

System Flow Chart



1



UI (Frontend)

- User logs in and interacts with the app
- Tracks workouts and hydration
- Starts battles and views progress

2



API (Backend)

- Handles all requests from frontend
- Authenticates users (JWT)
- Processes workouts and battle results

3



Database

- Stores user accounts and progress
- Saves workouts, XP, and coins
- Keeps battle history

4



AI System (Cloud API)

- Generates workout recommendations
- Responds to user questions
- Enhances gameplay experience



FitQuest Core APIs

API Name	Method	Endpoint	Purpose	Auth Required
Register User	POST	/register	Create new account	No
Login User	POST	/login	Authenticate and get token	No
Get Profile	GET	/profile	Retrieve user data	Yes
Log Workout	POST	/log-workout	Record workout, earn XP	Yes
Recommendations	GET	/api/recommendations	Get personalized workouts	Yes
AI Coach	POST	/api/ai-coach	Chat with fitness AI	Yes
Health Check	GET	/health	Verify server status	No

API - Register

Type: Post (RESTful)

Parameters:

- Email
- Password
- Username

Response:

Creates user

"success: true

Message: user registered"

Purpose:

The purpose of this API is to create and post a user's account to store in the database. By creating an account, users can access the dashboard by signing in using the email and password. The username is what the app is going to call the user. Their information is stored and used by our AI coach as well as our AI workout recommender. This creates a dashboard that is strictly their own with information corresponding to their stats.

API's

Register

https://comp-490-team-run-production.up.railway.app/register

POST https://comp-490-team-run-production.up.railway.app/register

Docs Params Authorization Headers 8 Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "email": "presentation@test.com",
3   "password": "123123"
4 }
```

Body Cookies Headers 11 Test Results 200 OK

{ } JSON Preview Visualize

```
1 {
2   "success": true,
3   "message": "User registered"
4 }
```

API Error Testing

Register Fail: Email already in use

The screenshot shows a REST client interface with the following details:

- URL:** `https://comp-490-team-run-production.up.railway.app/register`
- Method:** `POST`
- Body Type:** `raw` (selected), with `JSON` as a hint.
- Request Body:**

```
1 {  
2   "email": "professor@test.com",  
3   "password": "123123"  
4 }
```
- Response Status:** `400 Bad Request`
- Response Body:**

```
1 {  
2   "success": false,  
3   "message": "Email already exists"  
4 }
```

API - Login

Type: Post (RESTful)

Parameters:

- Email
- Password

Response:

Logs user into their account

"success: true

Message: successfully
logged in

Token: 'example token'

Stats: 'example stats' "

Purpose:

The purpose of this API is to log the user into their own personalized dashboard where they can start the game. The API is called and checks with the database to confirm whether or not credentials match. Failure will result in a message explaining the reason for the error. When successful a token is generated which gives the user permission to continue with their account and saves their progress as well. After a certain amount of time that token will expire and the user will need to login again. Same thing if they choose to sign out.

API's Login

 http://localhost:4000/login

POST

http://localhost:4000/login

Send

Docs

Params

Authorization

Headers 8

Body

Scripts

Settings

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

JSON

Schema

Beautify

```
1 {
2   "email": "test@test.com",
3   "password": "123123"
4 }
```

Body

Cookies

Headers 8

Test Results

200 OK

111 ms

637 B

{}

JSON

Preview

Visualize

```
1 {
2   "success": true,
3   "message": "Login successful",
4   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFnZW50IjoiI20TjZmNlZTRkMjgyM2IyNmNkMGNhZDMiLCJpYXQiOiE3NzgwNDA0NjEsImV4cCI6MTc3ODAwNzY2MX0.Dw_7VfMvi-tuQaxS9l0SRre4xHa330FlJ3PVC_ApPTk",
5   "heroName": "bruh",
6   "stats": {
7     "strength": 61,
8     "stamina": 61,
9     "agility": 33,
10    "health": 219
11  },
12   "level": 55,
13   "gold": 2160
14 }
```


API Error Testing

Login Fail: Incorrect Credentials

The image displays two side-by-side screenshots of a REST client interface, likely Postman, showing a failed login attempt. Both screenshots are for a POST request to `http://localhost:4000/login`.

Left Screenshot:

- Request Body (raw):**

```
1 {
2   "email": "test@test.com",
3   "password": "12312"
4 }
```
- Response (JSON):**

```
1 {
2   "success": false,
3   "message": "Incorrect password"
4 }
```
- Status Bar:** 400 Bad Request · 119 ms · 324 B

Right Screenshot:

- Request Body (raw):**

```
1 {
2   "email": "test@test.co",
3   "password": "123123"
4 }
```
- Response (JSON):**

```
1 {
2   "success": false,
3   "message": "User not found"
4 }
```
- Status Bar:** 400 Bad Request · 49 ms · 320 B

API - AI Coach

Type: Post (RESTful)

Calls <https://api.groq.com/openai/v1>

Parameters:

- Token
- Message

Response:

"success: true

Reply: 'sample reply'

Mode: 'groq'

Model: 'llama-3.3'

"

Purpose:

This API calls Groq which is an external cloud service that powers our AI coach. A user token is needed to interact with the AI since the coach directly correlates its reply based on your message, stats, and log in time. All this info is stored in the database, and the AI coach is only given access to what is needed to give a proper response, NO PERSONAL INFO IS GIVEN TO THE AI. The coach will give advice on what you could do better or even recommend diets to improve your game experience.

The screenshot shows a REST client interface with the following details:

- URL:** `https://comp-490-team-run-production.up.railway.app/api/ai-coach`
- Method:** `POST`
- Body (raw):**

```
1 {
2   "message": "What workouts should I do to get stronger?"
3 }
```
- Response (JSON):**

```
1 {
2   "success": true,
3   "reply": "What's up bruh 🤖! To boost your Strength, focus on compound exercises like Squats (you're already crushing those 💪) and add some Deadlifts, Bench Press, or Rows to your routine. These exercises will help increase your Strength stat, currently at 61. Aim for 2-3 sets of 8-12 reps. Don't forget to rest and recover, bruh! You've got this! 🍌",
4   "mode": "groq",
5   "model": "llama-3.3-70b"
6 }
```
- Status:** `200 OK` • 827 ms • 785 B

API's

Cloud AI API- AI COACH

Using groq : the app intakes questions given by the user and responds with advice based on the user's character on what to focus on for the day.

Error Response – Invalid Token

The screenshot displays a REST client interface with a sidebar on the left containing a list of collections. The main panel shows a POST request to `https://comp-490-team-run-production.up.railway.app/api/ai-coach` with Bearer Token authorization. The response is a 401 Unauthorized status with a JSON body containing the message: `"message": "Invalid or expired token"`.

Left Sidebar:

- COLLECTIONS
 - My Collection
 - GET Get data
 - POST Post data
 - POST register
 - POST login
 - GET health
 - GET ai recommendations
 - POST ai coach
 - POST https://summarize-lemon.vercel.app/api/summarize
 - GET BitByBit render
 - GET BitByBit games api
 - POST BitByBit ai api
- ENVIRONMENTS
- SPECS
- FLOWS

Main Panel:

- Overview | POST ai coach | No environment
- My Collection > ai coach
- POST | https://comp-490-team-run-production.up.railway.app/api/ai-coach | Send
- Docs | Params | Authorization | Headers 11 | Body | Scripts | Settings | Cookies
- Auth Type: Bearer Token | Token: [Redacted]
- The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.
- Body | Cookies | Headers 10 | Test Results | 401 Unauthorized • 447 ms • 399 B • Save Response
- { } JSON | Preview | Pass the correct auth credentials

```
1 {
2   "message": "Invalid or expired token"
3 }
```

Right Sidebar:

- New Chat
- Describe what you need. Press @ for context.
- Auto

Bottom Bar:

- Connect Git | Console | Terminal
- Cloud Agent | Globals | Vault | Tools

API - Workout Recommendation (local AI)

Type: Get (RESTful)

Parameters:

- Token

Response:

"success: true

Message: 'sample
message'

Reply: 'sample
recommendation'
"

Purpose:

This API gathers data from the user's least leveled stat as well as last time they logged in to determine what workouts the user should do next. Instead of having to ask the coach, this feature is fast and straight to the point no extra steps needed. This simple AI follows these simple rules find weakest stat, neglected exercises, and suggest targeted workouts. Having a simple rule-based AI showed quicker results than always relying on generative AI Groq.

API's

API- Recommendation

Recommends workouts
to users based on
character levels and
weaknesses

5/14/2026

https://comp-490-team-run-production.up.railway.app/api/recommendations

GET https://comp-490-team-run-production.up.railway.app/api/recommendations

Auth Type: Bearer Token

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Token: AYI2YWtlm2nbVkCx7r9yDvmsm-bGCK

Body: 200 OK • 59 ms • 1.63 KB

```
{
  "success": true,
  "motivationalMessage": "🔥 Your character misses you! Time to level up!",
  "recommendations": [
    {
      "id": "target_weakness",
      "name": "Agility Booster",
      "exercises": [
        {
          "name": "Jumping Jacks",
          "reps": 52,
          "unit": "reps",
          "sets": 3,
          "icon": "🔥",
          "description": "Full body cardio"
        }
      ]
    },
    {
      "name": "High Knees",
      "reps": 47,
      "unit": "reps",
      "sets": 3,
      "icon": "🔥",
      "description": "Speed and coordination"
    },
    {
      "name": "Lateral Lunges",
      "reps": 39,
      "unit": "reps",
      "sets": 3,
      "icon": "🔥"
    }
  ]
}
```

Error Response – Invalid Token

The screenshot displays a REST client interface with a sidebar on the left containing a list of collections. The main panel shows a GET request to `https://comp-490-team-run-production.up.railway.app/api/recommendations` with Bearer Token authorization. The response is a 401 Unauthorized status with a JSON body containing the message: `"message": "Invalid or expired token"`. The interface includes tabs for Overview, Authorization, Headers, Body, Scripts, and Settings. The bottom status bar shows the date 07/14/2020 and various tool icons.

Left Sidebar:

- COLLECTIONS
 - My Collection
 - GET Get data
 - POST Post data
 - POST register
 - POST login
 - GET health
 - GET ai recommendations
 - POST ai coach
 - POST https://summarize-lemon.vercel.app/api/summarize
 - GET BitByBit render
 - GET BitByBit games api
 - POST BitByBit ai api
- ENVIRONMENTS
- SPECS
- FLOWS

Main Panel:

Overview GET ai recommendations + No environment

My Collection > ai recommendations Save Share

GET https://comp-490-team-run-production.up.railway.app/api/recommendations Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Auth Type: Bearer Token Token: [Redacted]

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body Cookies Headers 10 Test Results 401 Unauthorized • 501 ms • 399 B • Save Response

{ JSON Preview Pass the correct auth credentials

```
1 {
2   "message": "Invalid or expired token"
3 }
```

Right Sidebar:

New Chat

Describe what you need. Press @ for context.

Auto

Bottom Bar:

Connect Git Console Terminal 07/14/2020 Cloud Agent Globals Vault Tools

API - Health

Type: Get (RESTful)

Parameters:

None

Response:

```
" status: 'healthy'  
  timestamp: '2026-05-  
13T07:06:39.354Z'  
  service: 'FitQuest API'  
"
```

Purpose:

Although this seems like a simple API, it plays a big role in which Railway (our cloud deployment) pings this API to check whether the server is still running. If it is then it will continue to run but if it isn't then railway will automatically restart after a few failed attempts to reach API. This also gives the team a quick way to find out if the server is still healthy and if its not we know something went wrong when booting.

API - Health

DEVELOPMENT & DEPLOYMENT

- **Development:**
- Localhost testing
- GitHub version control

Deployment:

- Hosted on Railway GCP(Google Cloud Platform)
- MongoDB Atlas (NOSQL cloud database)



Roadblocks

- Lack of Experience

The team lacked experience with AI integration so trying to find ways to implement it was difficult.

- API Key Leak

Our Gemini API key was pushed onto the main git which resulted in the immediate shut off. For a while we could not understand why the ai coach was failing.

- Time Conflict

Team Run found it hard to find good opportunities to meet up since many of us have work and other priorities to attend to.

- Switching Gemini to Groq

At some point during the transition from local to cloud hosting, Gemini became incompatible with the cloud hosting resulting in alternative

- Missing dependency

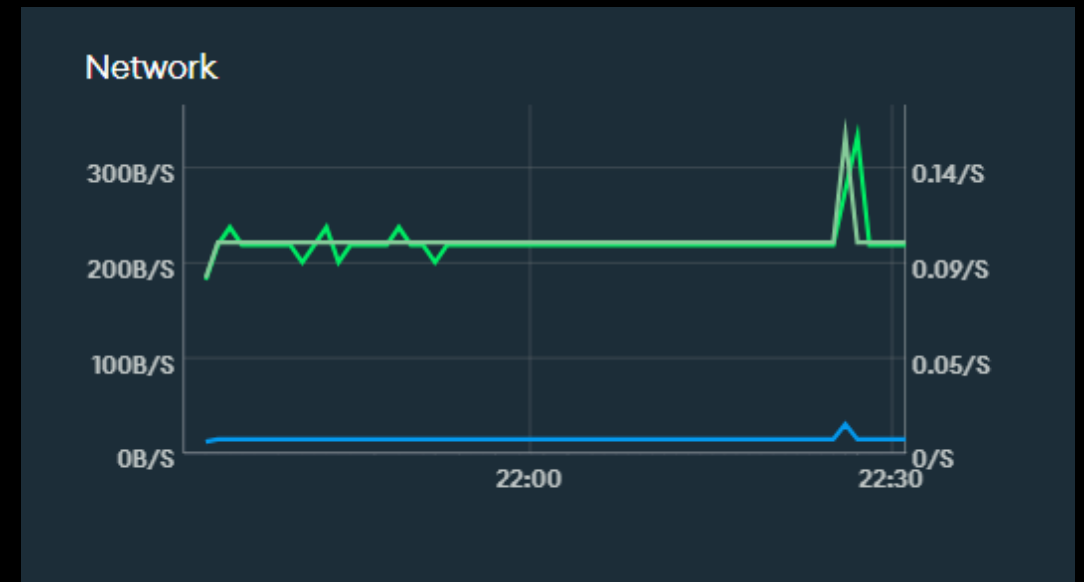
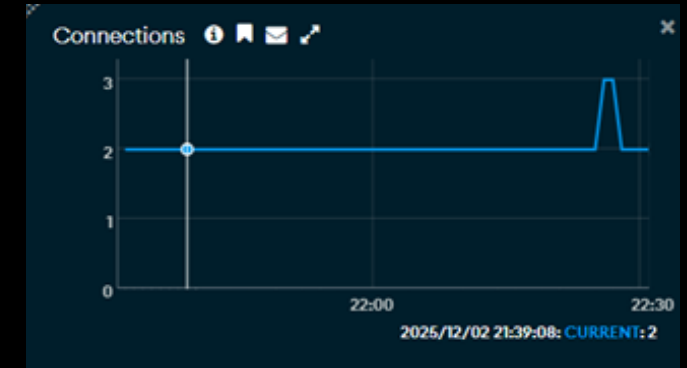
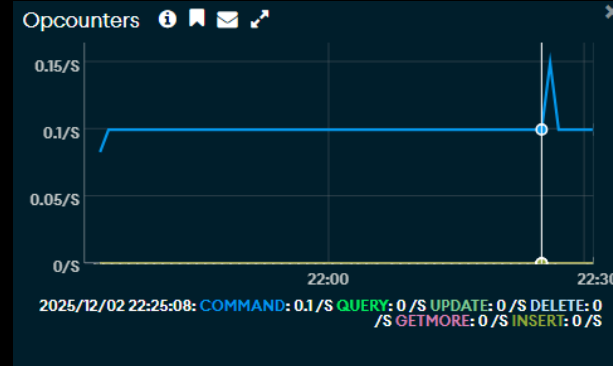
@google/generative-ai not in package.json which caused AI failure. Fallback mode was implemented as a result just in case issues with AI arose

- Cloud Based Database & Hosting

Since we already had a set system the previous semester, having to switch from local hosting with a local database to a cloud database and cloud hosting meant changing backend logic

Metrics & Achievements

- **Performance Comparison**
- During testing, we noticed faster loading times and smoother updates around night compared to mornings.
- **Response Time Improvement**
- We monitored our deployed backend and saw consistent response activity with no crashes while sending multiple requests.
- **Feature Implementation Rate**
- We have the main features:
- Account creation and login
- Secure user authentication
- Workout and water logging that saves to the database
- We focused on getting the core functionality done before moving on to extra features.



Future Enhancements

- Mobile app version
- Social features (compete with friends)
- More enemy variety and boss battles
- Achievement badges and rewards
- Integration with fitness wearables

Project Completion Checklist

APIs Implemented

- ✓ Login API
- ✓ Register API
- ✓ Battle Result API
- ✓ AI Coach API (Cloud API)

GitHub

[- Github-Team-Run](#)

Core System

- ✓ UI (Frontend)
- ✓ API (Backend)
- ✓ Database (MongoDB Atlas)
- ✓ AI System (Cloud API)
- ✓ Full System Flow Connected

This slide shows everything we successfully completed for the project.

We implemented the full system, built multiple APIs including a cloud AI API, tested everything in Postman and deployed our backend and database.

Testing & Validation

- ✓ Postman Testing
- ✓ Successful API Responses
- ✓ Error Handling Tested
- ✓ Peer's API Testing Completed

Cloud Integration

- ✓ Backend Deployed (Railway)
- ✓ Database Hosted (MongoDB Atlas)
- ✓ AI Cloud API Integrated

Features Completed

- ✓ Workout Tracking
- ✓ XP & Coins System
- ✓ Battle System
- ✓ Water Intake Tracker
- ✓ AI Recommendations